

CYBER SECURITY: LABS

LAB 2B: PKI OPENSSEL (LINUX)

BY D.E. MENACER (CEH v12, CEH v13 with AI, PCEH, MCSE)

COPYRIGHT © ESI – 2026

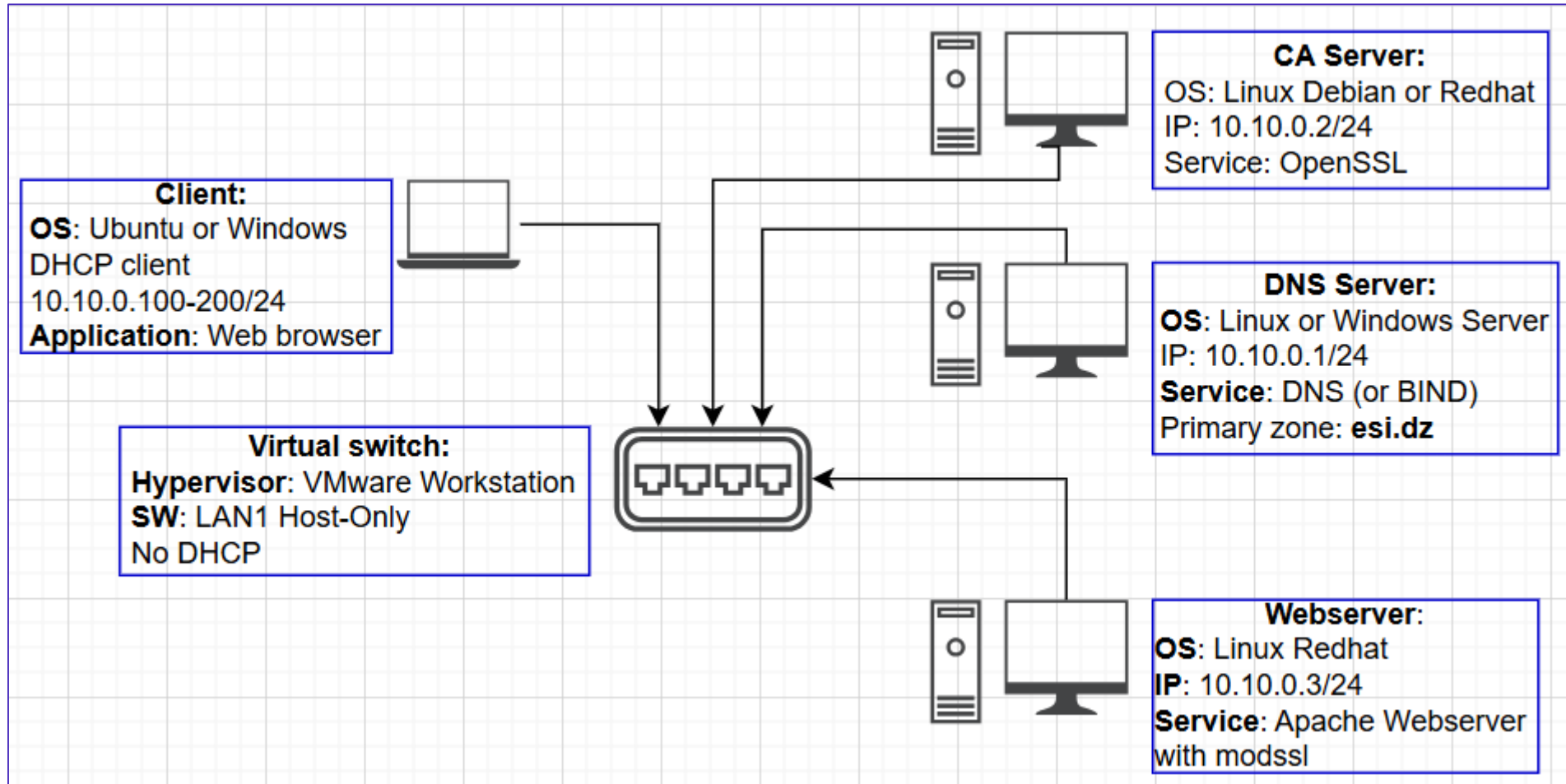
OVERVIEW

- 1. Objectives**
- 2. Lab Environnement**
- 3. Preparation**
- 4. HTTP site creation**
- 5. HTTP test and Wireshark analysis**
- 6. CA Creation with OpenSSL**
- 7. WebServer Certificate Creation**
 - a. modssl setup**
 - b. Apache configuration**
- 8. SSL Certificat setup**
- 9. HTTPS site test and Wireshark analysis**

OBJECTIVES

- The main objective of this Lab is to create a **certificate authority** and manage **SSL digital certificates** for use on an Apache web server under Linux.
- The tool used will be **OpenSSL**.
- Students will create first a non-secure website (**http**). Then with a **server certificate**, they configure Apache Server to make the website secure (**https**).

VM ENVIRONMENT



PREPARATION: ON CA SERVER VM

- Using **root** privileges by **sudo** or **su** -
- **Working directories**: To store certificates, public and private keys, the following directories must be created :
- **\$HOME=/root/ca**
 - mkdir \$HOME/certs
 - mkdir \$HOME/crl
 - mkdir \$HOME/private
- ***openssl.cnf*** : configuration file used by the ***openssl*** commands.
- Copy **/etc/pki/tls/openssl.cnf** in **\$HOME**

PREPARATION

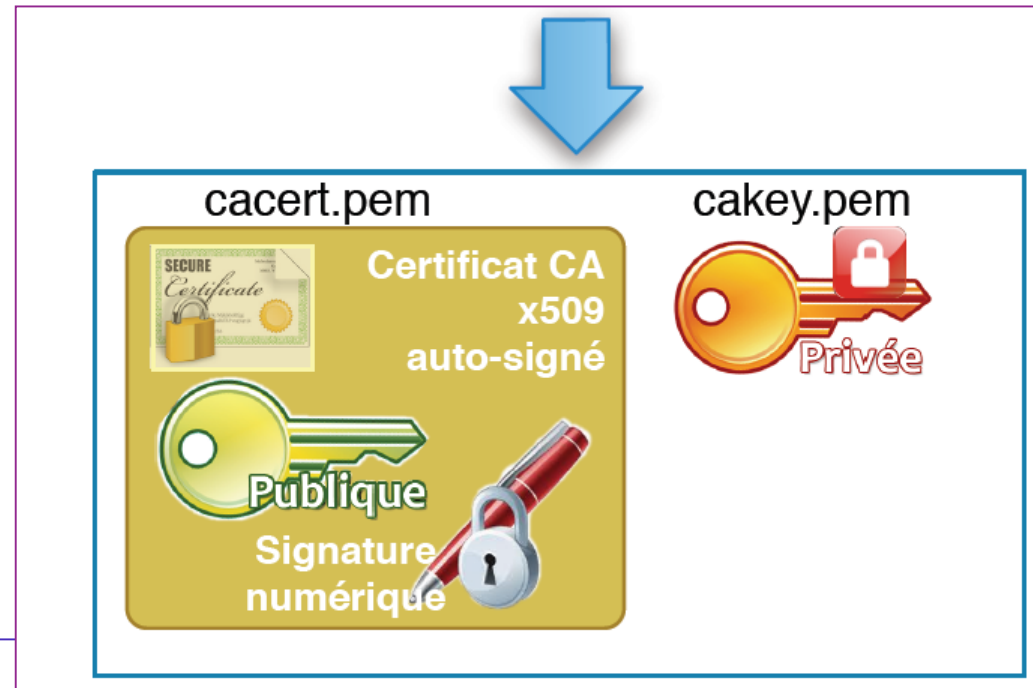
- In **openssl.cnf**, change the **\$DIR** variable to the previous **\$HOME**.
- Create the following files in **/root/ca**:
- ***index.txt***: certificates database
 - # touch \$HOME/index.txt
- ***serial***: contains the **serial number** of the last certificate created, incremented with each certificate creation.
 - # echo 01 > \$HOME/serial

CREATION OF THE CERTIFICATION AUTHORITY

- ***Root Certificate***: First, a private-public key pair must be created, then a self-signed root certificate (signed by the certificate itself). We will have :
 - A password-protected private key (`cakey.pem`)
 - A request for a digital certificate valid for 3650 days (`cacert.pem`)

CREATION OF THE CERTIFICATION AUTHORITY

```
# openssl req -new -x509 -extensions v3_ca -keyout  
$HOME/private/cakey.pem -out $HOME/cacert.pem -days 3650 -config  
$HOME/openssl.cnf
```



INFORMATION TO PROVIDE

Field	Description	Example
PEM passphrase	Password used to encrypt the private key	12345678
Country Name	The two letters of the country where the certificate was created	DZ
State or Province Name	Region or department	ALGIERS
City or Locality	City	ALGIERS
Organization Name	Company	ESI
Organizational Unit	Organizational unit	IT
Common Name	Descriptive name of the certificate	ESI CA

CHECKING THE ROOT CERTIFICATE

- The private key is protected with a variant of the 3DES algorithm. The entered password will be required to read the key.
- Displaying the root certificate :

```
# openssl x509 -text -in $HOME/cacert.pem
```

```
# chmod 600 $HOME/private/cakey.pem
```

```
# chmod 644 $HOME/cacert.pem
```

- **File backup:** To avoid accidental deletion of the root certificate or, worse, the private key, it is necessary to duplicate (archive) the two files:

```
# tar -czf rootca.tar.gz private/cakey.pem cacert.pem
```

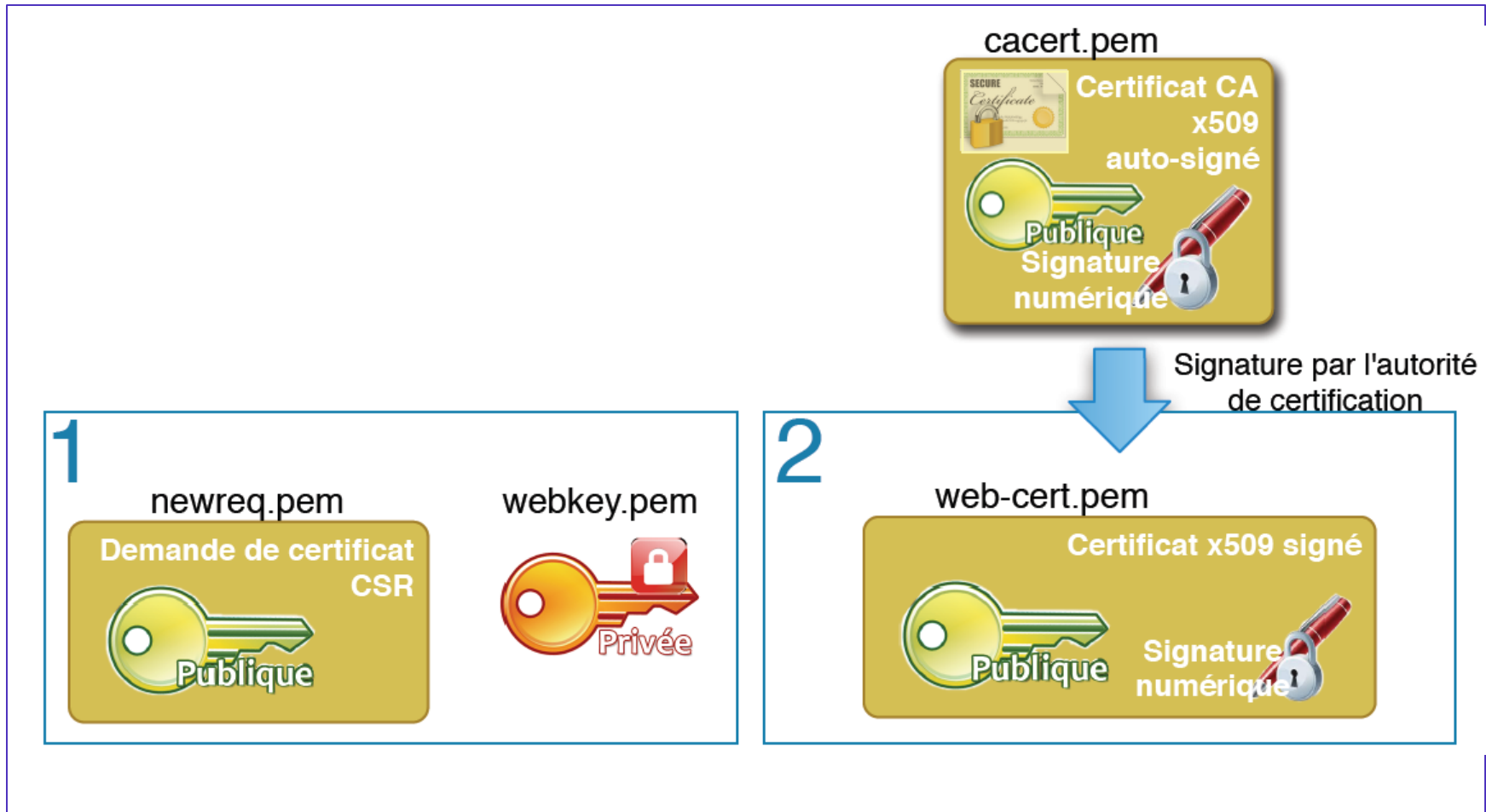
CREATING AN SSL CERTIFICATE FOR A WEB SERVER

- An **SSL certificate** can be useful for securing communication between a **web server** and potential clients (ex. **Web browser**).
- This certificate, installed on a **web server**, is sent to the client when a secure connection is requested.
- The SSL certificate, along with a **public/private key pair**, allows the server to exchange encrypted data with the client's browser.

WEB SERVER CERTIFICATE CREATION PROCESS

- Create a new public/private key pair (**webkey.pem**).
- Create a new certificate request for the server that will contain the public key (**newreq.pem**)
- Sign this certificate application with the certificate from the authority (**cacert.pem**) and obtain a new certificate (**webcert.pem**).

WEB SERVER CERTIFICATE CREATION PROCESS



CREATING THE KEY PAIR AND THE CERTIFICATE REQUEST

```
# openssl req -config $HOME/openssl.cnf -new -keyout  
$HOME/private/webkey.pem -out $HOME/certs/newreq.pem
```

CREATING THE KEY PAIR AND THE CERTIFICATE REQUEST

Champ	Explication	Exemple
PEM passphrase	Password used to encrypt the private key	123456
Country Name	The two letters of the country where the certificate was created	DZ
State or Province Name	Region or department	ALGIERS
City or Locality	City	ALGIERS
Organization Name	Company	ESI
Organizational Unit	Organizational unit	IT
Common Name	Descriptive name of the certificate. Must represent the fully qualified DNS name of the server.	www.esi.dz

SIGNATURE OF THE CERTIFICATE APPLICATION BY THE AUTHORITY

- This certificate now needs to be **signed** so that it can be deployed on the web server.
- For this, the **certificate authority's private key** will be required, as it is the **only key that can create the digital signature**.

SIGNATURE OF THE CERTIFICATE APPLICATION BY THE AUTHORITY

FIELD	DESCRIPTION	EXAMPLE
PEM passphrase for root key	Password to decrypt the certificate authority's private key. The private key will be used to create the Signature.	12345678
Sign the certificate? [y/n]	Do we need to sign the certificate?	y
1 out of 1 certificate Requests certified, commit? [y/n]y	Submit the request?	y

SIGNATURE OF THE CERTIFICATE APPLICATION BY THE AUTHORITY

```
# openssl ca -config $HOME/openssl.cnf -policy policy_anything  
-out $HOME/certs/webcert.pem -infiles  
$HOME/certs/newreq.pem
```

- **Permissions**

```
# chmod 600 $HOME/private/webkey.pem  
# chmod 644 $HOME/certs/webcert.pem
```

CERTIFICATION PATH VERIFICATION

- The goal is to **verify** that the certificate was indeed signed by the certification authority.
- This proves that the certification path is correct.
- ***Verify*** command :

```
# openssl verify -CAfile $HOME/cacert.pem  
$HOME/certs/webcert.pem
```

```
→ certs/webcert.pem: OK
```

CHECKING THE WEBSERVER CERTIFICATE

- Display the webserver certificate

```
# openssl x509 -text -in $HOME/certs/webcert.pem
```

- Unprotect the webserver private key

```
# openssl rsa -in $HOME/private/webkey.pem -out  
$HOME/private/webkey-clear.pem
```

INSTALLING THE SSL CERTIFICATE

- *Exporting certificates and private key*
- *Creating websites*
- *Configuring Apache*
- *Adding the new authority to the browser*

EXPORTING CERTIFICATES AND PRIVATE KEY

- The following elements are required for Apache Web Server to support SSL. :
 - The server certificate (webcert.pem)
 - The server's unencrypted private key

CREATING WEBSITES: WEBSERVER VM

- Create the following websites :
 - <http://www.esi.dz>
 - <https://www.esi.dz>
- **NB.** *We are in private mode (Intranet), so there is no interference with ESI's public sites..*

CONFIGURING APACHE WEBSERVER

- Installing Apache web server if necessary:

```
# yum install httpd
```

- Installing **ssl module** for apache

```
# yum install mod_ssl
```

- Normally, the following files exist:

- /etc/httpd/conf.d/vhost.conf

- /etc/httpd/conf.d/ssl.conf

NON SECURE WEBSITE: <http://www.esi.dz>

- Create a webpage `index.html` (with any editor) in `/var/www/html/esi/site80`
`chown apache index.html`
`chgrp apache index.html`
- Add the http service in firewalld:
`firewall-cmd --add-service=http`
- restart httpd and firewall services :
`systemctl restart httpd`
`systemctl restart firewalld`

<http://www.esi.dz>

- **Edit the file `/etc/httpd/conf.d/vhost.conf`**

`<VirtualHost *:80>`

`ServerAdmin webmaster@esi.dz`

`DocumentRoot /var/www/html/esi/site80`

`ServerName www.esi.dz`

`ErrorLog logs/www.esi.dz-error_log`

`CustomLog logs/www.esi.dz-access_log common`

`</VirtualHost>`

WEBSITE TEST: ON CLIENT VM

- Open the web browser [Firefox or Chrome]:
 - Type: <http://www.esi.dz>
- The website should be working.
- **Bonus** : with **Wireshark**, show that all the packets exchanged between the client and the webserver are sent **unsecure**.

SECURE WEBSITE <https://www.esi.dz> CREATION: ON WEBSERVER VM

- Create a webpage `index.html` (with any editor) in `/var/www/html/esi/site443`
`chown apache index.html`
`chgrp apache index.html`
- Add the `https` service in `firewalld`:
`firewall-cmd --add-service=https`
- restart `httpd` and `firewalld` services :
`systemctl restart httpd`
`systemctl restart firewalld`

SECURE WEBSITE <https://www.esi.dz>

SSL VIRTUAL HOST CONFIGURATION

- **# vi /etc/httpd/conf.d/ssl.conf**

<VirtualHost *:443>

ServerName www.esi.dz

DocumentRoot /var/www/esi/site443

SSLEngine on

SSLCertificateFile /root/ca/certs/webcert.pem

SSLCertificateKeyFile /root/ca/private/webkey-clear.key

</VirtualHost>

SECURE WEBSITE <https://www.esi.dz>

CHECKING SSL CONFIGURATION

```
# httpd -t
```

Syntax OK

```
# apachectl configtest
```

Syntax OK

```
# httpd -D DUMP_VHOSTS
```

```
# systemctl restart httpd --optional
```

- Check the SSL connection

```
# openssl s_client -connect www.esi.dz:443 --state
```

DNS CONFIGURATION : ON DNS SERVER

- Install the BIND service (Linux) or DNS role (Windows Server).
- Create primary zone esi.dz
- Create host's records (A record) for all VMs
- Create an alias [www.esi.dz](#) (CNAME record) pointing to [webserver.esi.dz](#)

SECURE WEBSITE <https://www.esi.dz>
FINAL CONFIGURATION: ON CLIENT VM

- Add the **ESI CA certificate** in the Firefox browser
- Be sure that timers in all VMs are synchronized (use NTP servers if needed)
- Test the **naming connectivity** with **ping**:
 - ping 10.10.0.3
 - ping www.esi.dz

SECURE WEBSITE <https://www.esi.dz>

THE TEST: ON CLIENT VM

- From the browser, Firefox ou Chrome:
<https://www.esi.dz>
- The site should display normally.
- View the **certificate** in your browser.
- Verify the **cryptographic** information.
- **Bonus:** with **Wireshark**, show that all the packets exchanged between the client and the webserver are sent **secure**.